

♥ Like

📌 Save

🔗 Share



Eric Hu

JavaScript Events



Adding event listeners

Use `addEventListener` to attach a function to an event.

js



```
1 const button = document.getElementById("submit");
2
3 button.addEventListener("click", () => {
4   console.log("Button clicked");
5 });
```

You can also use `onevent` properties, but `addEventListener` is preferred because it supports multiple listeners.

js



```
1 button.onclick = () => {
2   console.log("Clicked");
3 };
```

Common event types

Event type	Fires when...
<code>click</code>	An element is clicked.
<code>dblclick</code>	An element is double-clicked.
<code>mouseover</code>	The pointer enters an element.
<code>mouseout</code>	The pointer leaves an element.
<code>keydown</code>	A key is pressed down.
<code>keyup</code>	A key is released.
<code>submit</code>	A form is submitted.
<code>change</code>	The value of an input changes.
<code>input</code>	The value of an input changes live.
<code>focus</code>	An element receives focus.
<code>blur</code>	An element loses focus.
<code>load</code>	A resource finishes loading.
<code>resize</code>	The viewport is resized.
<code>scroll</code>	An element is scrolled.

The event object

The listener receives an event object with useful properties and methods.

js



```
1 document.addEventListener("click", (event) => {
2   console.log(event.type); // "click"
3   console.log(event.target); // element that was clicked
4   console.log(event.clientX); // horizontal click
position
5   console.log(event.clientY); // vertical click position
6 });
```

Common event object properties.

Property	Description
target	The element that triggered the event.
currentTarget	The element the listener is attached to.
type	The event type.
key	The key pressed for keyboard events.
preventDefault()	Cancels the default action.
stopPropagation()	Stops event propagation.

Event propagation

Events travel through the DOM in two phases: bubbling and capturing.

Bubbling is when the event travels from the child to the parent.

js



```
1  const parent = document.getElementById("parent");
2  const child = document.getElementById("child");
3
4  parent.addEventListener("click", () => {
5    console.log("Parent clicked");
6  });
7
8  child.addEventListener("click", () => {
9    console.log("Child clicked");
10 });
11
12 // Console logs ⬇
13 // Child clicked
14 // Parent clicked
```

Capturing is when the event travels from the parent down to the child element.

Set the third argument of `addEventListener` to `true` to listen during the capturing phase.

js



```
1  const parent = document.getElementById("parent");
2  const child = document.getElementById("child");
3
4  parent.addEventListener(
5    "click",
6    () => {
7      console.log("Parent clicked");
8    },
9    true,
10 );
11
12 child.addEventListener(
13   "click",
14   () => {
15     console.log("Child clicked");
16   },
17   true,
18 );
19
20 // Console logs ⬇
21 // Parent clicked
22 // Child clicked
```

Event delegation

Attach one listener to a parent to handle events from many children.

js



```
1 document.getElementById("list").addEventListener("click",  
(event) => {  
2   if (event.target.tagName === "LI") {  
3     console.log(event.target.textContent);  
4   }  
5 });
```

This works for dynamically added elements too.

Preventing default behavior

Use `preventDefault` to stop the browser's default action.

js



```
1 document.querySelector("a").addEventListener("click",  
(event) => {  
2   event.preventDefault();  
3   console.log("Link click prevented");  
4 });
```

Common use cases include preventing form submissions and link navigation.

js



```
1 document.querySelector("form").addEventListener("submit",  
(event) => {  
2   event.preventDefault();  
3   // validate and submit with fetch  
4 });
```

Stopping propagation

Use `stopPropagation` to prevent the event from reaching parent elements.

js



```
1 child.addEventListener("click", (event) => {  
2   event.stopPropagation();  
3   console.log("Child only");  
4 });
```

Removing event listeners

Use `removeEventListener` to detach a listener.

js



```
1 function handleClick() {  
2   console.log("Clicked");  
3 }  
4  
5 button.addEventListener("click", handleClick);  
6 button.removeEventListener("click", handleClick);
```

Keyboard events

Listen for key presses and inspect the `key` property.

js



```
1 document.addEventListener("keydown", (event) => {  
2   if (event.key === "Enter") {  
3     console.log("Enter pressed");  
4   }  
5 });
```

Useful modifier properties.

js



```
1 document.addEventListener("keydown", (event) => {  
2   if (event.ctrlKey && event.key === "s") {  
3     event.preventDefault();  
4     console.log("Save shortcut");  
5   }  
6 });
```

Full-Stack Developer Roadmap

AI

From HTML & CSS to working with AI
models, all in one structured roadmap.



www.thedevspace.io